VPC PROJECT

Implementation of Multi-AZ Scalable Infrastructure
using AWS VPC, ALB, and NAT Gateway

IMPLEMENTATION GUIDE
Step-by-step configuration based on the supplied project report

Project flow

```
VPC & Subnets → Route Tables → IGW / NAT Gateway
EC2 Website → AMI → Launch Template
ALB & Target Group → Auto Scaling Group → Stress Test
```

Prepared from the steps and settings in the original VPC Project report.

1. Components Used

- Public and private subnets
- Route tables
- Internet Gateway
- NAT Gateway
- EC2 instances
- Auto Scaling Group
- Elastic Load Balancer

2. Create the VPC, Subnets, and Route Tables

Create a VPC

- Open the AWS VPC service and choose Create VPC.
- Enter a VPC name and set the CIDR block to **10.0.0.0/16**.
- Create the VPC.

Create four subnets

Create the following subnets using the CIDRs and Availability Zones listed in the source report.

SUBNET NAME	SUBNET CIDR	AVAILABILITY ZONE
public-subnet-01	10.0.0.0/22	ap-northeast-3a
public-subnet-02	10.0.4.0/22	ap-northeast-3b
private-subnet-01	10.0.8.0/22	ap-northeast-3a
private-subnet-02	10.0.12.0/22	ap-northeast-3b

Create two route tables

- Create one public route table and one private route table.
- Associate both public subnets with the public route table.
- Associate both private subnets with the private route table.

3. Create and Attach the Internet Gateway

- In the VPC dashboard, create an Internet Gateway.
- Attach the Internet Gateway to the created VPC.
- Open the public route table and edit routes.
- Add destination **0.0.0.0/0** with the created Internet Gateway as the target, then save.

4. Create the NAT Gateway

- In the VPC dashboard, create a NAT Gateway in a public subnet.
- Select the zonal option and allocate an Elastic IP.
- Create the NAT Gateway and wait for it to become available.
- Open the private route table and edit routes.
- Add destination **0.0.0.0/0** with the created NAT Gateway as the target, then save.

5. Launch an EC2 Instance and Host the Website

- Launch an Ubuntu EC2 instance in the default VPC, as described in the report.
- Allow HTTP (80) and SSH (22) in its security group.
- Connect to the instance.

Install Nginx and download a website template

```
sudo apt update
sudo apt install unzip
sudo apt install nginx
wget -O app.zip <download-link-of-template>
unzip app.zip
```

- Copy the extracted website files to the Nginx web directory.

```
sudo cp -r <extracted-folder-name>/* /var/www/html
```

Verify the website in a browser using http://<public-IP-instance>:80.

6. Create an AMI

- Select the EC2 instance hosting the application.
- Choose Actions → Images and templates → Create image.
- Enter an image name and create the image.
- Open AMIs and verify that the image status is **Available**.

7. Create a Launch Template

- Open Launch Templates and choose Create launch template.
- Enter a template name and select the Auto Scaling guidance option.
- Select the created image under My AMIs.
- Choose instance type **t2.micro** and create/select a key pair.
- Under network settings, select the security group associated with the created VPC.
- Create the Launch Template.

8. Create the Auto Scaling Group and Load Balancer

- Open Auto Scaling Groups and choose Create Auto Scaling group.
- Enter a name and select the created Launch Template.
- Select the two private subnets in different Availability Zones.
- Attach a new Application Load Balancer and select **Internet-facing**.
- Select the public subnets for the Load Balancer.
- Create a target group and configure the listener.
- Set desired capacity to **1**, minimum capacity to **1**, and maximum capacity to **3**.
- Choose target tracking scaling policy, select CPU utilization, and set target value to **5**.
- Create the Auto Scaling Group and confirm that the Load Balancer is active.

Open the Load Balancer DNS endpoint in a browser and verify that the application is accessible.

9. Launch a Public EC2 Instance for SSH Access

- Launch an Ubuntu **t2.micro** instance in a public subnet of the created VPC.
- Attach the created key pair.
- Select the created VPC and a public subnet, and enable a public IP.
- Select the security group for the created VPC and launch the instance.

10. SSH into a Private Instance

- Connect to the public EC2 instance.
- Create a **key.pem** file containing the private key content for the key pair used by the private instance, as described in the report.
- Set the file permission.

```
chmod 400 key.pem
```

The source report gives this SSH command:

```
ssh-keygen -i "key.pem" ubuntu@<private-IP-desired-instance>
```

Use the SSH command format appropriate to your client.

11. Generate CPU Load and Observe Scaling

```
sudo apt update && sudo apt install stress -y
stress --cpu 2 --timeout 300
```

- Run the stress command on the desired private instance.
- Wait a few minutes and observe the EC2 dashboard.
- Check whether the Auto Scaling Group launches additional instances.

12. Project Verification

- VPC resource map shows the VPC, four subnets, route tables, and Internet Gateway.
- The application is accessible through the Load Balancer DNS endpoint.
- The Auto Scaling Group has the configured desired, minimum, and maximum capacities.
- After CPU stress, check the EC2 instance list to see whether scaling occurred.